

Guia de Exposicion

Cambios en Frontend y Backend: que es, que hace, para que sirve

Cada cambio incluye: la **ruta exacta** del archivo (para abrir la carpeta y el codigo), una ficha **Que es / Que hace / Para que sirve**, y el **codigo exacto** tal como quedo en el proyecto.

Proyecto	Carpeta raiz	Puerto
Backend (API + contratos)	E:\Portafolio\mod_02_productos	3000
Frontend (dashboard Next.js)	E:\Portafolio\frontend_productos	3001
Blockchain local (Ganache GUI)	http://127.0.0.1:7545 (chainId 1337)	7545

Cuentas de Ganache usadas (importadas en MetaMask):

Cuenta	Direccion	Rol
A (organizador)	0x2c6fA8191d825B787dD931E80b939d8FEE6EEed9	Firma deploy/registros; duena inicial de los NFT
B (comprador)	0xada53769c1679cc8A927806b7417Cc7B297Bcd9d	Destino de la transferencia del boleto NFT

Mapa general de los cambios

Idea central: el sistema ya registraba eventos on-chain (contrato Events) pero el "boleto NFT" no existia de verdad. Se porto el patron del Lab 12 (facturas NFT) para: (1) acunar un NFT ERC-721 por evento, (2) guardar su tokenId y dueño en MariaDB, (3) transferirlo entre cuentas de MetaMask/Ganache, y (4) validarlo leyendo la cadena.

#	Archivo (backend)	Tipo
B1	contracts\EventoNFT.sol	NUEVO - contrato ERC-721
B2	scripts\deploy.cjs	MODIFICADO - despliega 2 contratos
B3	src\blockchain\eventoNftContract.js	NUEVO - modulo web3 del NFT
B4	src\service\eventoService.js	MODIFICADO - guardar tokenId/dueno
B5	src\controllers\eventoController.js	MODIFICADO - 4 handlers NFT
B6	src\routes\eventoRoutes.js	MODIFICADO - 4 rutas NFT
B7	src\db\schema.sql	MODIFICADO - 2 columnas nuevas
B8	scripts\verificar-nft.cjs	NUEVO - prueba automatica

#	Archivo (frontend)	Tipo
F1	lib\nft.ts	NUEVO - mint / transferir / verificar
F2	lib\api.ts	MODIFICADO - 3 funciones nuevas
F3	lib\types.ts	MODIFICADO - campos nftTokenId/nftOwner
F4	lib\swal.ts	MODIFICADO - dialogo promptText
F5	components\dashboard\EventsTable.tsx	MODIFICADO - 3 botones NFT
F6	app\dashboard\page.tsx	MODIFICADO - pasa la wallet
F7	package.json	MODIFICADO - puerto 3001 (fix API)
F8	.env.local	MODIFICADO - direccion del contrato

Ademas: migracion en MariaDB (columnas nft_token_id y nft_owner) y re-registro on-chain de los 3 eventos en el contrato vigente para que la validacion coincida al 100%.

Contrato EventoNFT.sol (NUEVO)

RUTA: E:\Portafolio\mod_02_productos\contracts\EventoNFT.sol

Que es	Un contrato inteligente ERC-721 (el estandar de los NFT) escrito en Solidity, basado en OpenZeppelin 4.9.6.
Que hace	Acuna (mint) un boleto NFT unico por evento, guarda su metadata dentro de la cadena (nombre, lugar, precio, hash, fecha de acunacion) e impide duplicados. Expone getTokenId, getMetadata y las funciones estandar (ownerOf, safeTransferFrom).
Para que sirve	Es el boleto digital : un token con dueno verificable que se puede transferir entre wallets. Sin este contrato el sistema solo 'registraba datos', no emitia propiedad.

Codigo exacto (nucleo del contrato)

```
contract EventoNFT is ERC721, Ownable {
    uint256 private _tokenIdCounter;
    error NFTYaExiste(uint eventoId); // no acunar 2 veces el mismo evento
    error DireccionInvalida(); // no acunar a la direccion 0x0
    error CampoRequerido(string campo); // name/lugar obligatorios

    struct EventoMetadata {
        uint eventoId; string name; string lugar; string precio;
        bytes32 hash; uint256 timestamp;
    }
    mapping(uint256 => EventoMetadata) public eventoMetadata; // tokenId -> metadata
    mapping(uint256 => uint256) public eventoToTokenId; // eventoId -> tokenId

    function mintEventoNFT(uint eventoId, string memory name, string memory lugar,
        string memory precio, bytes32 hash, address to) public returns (uint256) {
        if (eventoToTokenId[eventoId] != 0) revert NFTYaExiste(eventoId);
        if (to == address(0)) revert DireccionInvalida();
        if (bytes(name).length == 0) revert CampoRequerido("name");
        uint256 tokenId = _tokenIdCounter++;
        _safeMint(to, tokenId); // crea el token y lo asigna a 'to'
        eventoMetadata[tokenId] = EventoMetadata(eventoId, name, lugar, precio,
            hash, block.timestamp);

        eventoToTokenId[eventoId] = tokenId;
        emit NFTMinted(tokenId, eventoId, to);
        return tokenId;
    }
}
```

B2

Script de despliegue deploy.cjs (MODIFICADO)

RUTA: E:\Portafolio\mod_02_productos\scripts\deploy.cjs

Que es	El script que sube (despliega) los contratos a Ganache usando Hardhat. Se ejecuta con npm run deploy .
Que hace	Antes desplegaba solo Events; ahora despliega Events + EventoNFT y guarda la direccion y el ABI de cada uno en dos JSON dentro de src\blockchain: contract-info.json (Events) y nft-info.json (EventoNFT).
Para que sirve	El backend y el frontend leen esos JSON para saber DONDE esta cada contrato, sin editar el codigo a mano tras cada deploy.

Codigo exacto (flujo principal)

```
async function main() {
  const [deployer] = await hre.ethers.getSigners();
  // 1) Contrato Events (registro de datos legibles on-chain)
  const eventsAddress = await desplegar('Events');
  guardarInfo('contract-info.json', eventsAddress, 'Events', ...);
  // 2) Contrato EventoNFT (boleto NFT ERC-721)
  const nftAddress = await desplegar('EventoNFT');
  guardarInfo('nft-info.json', nftAddress, 'EventoNFT', ...);
}
```

B3

Modulo blockchain del NFT eventoNftContract.js (NUEVO)

RUTA: E:\Portafolio\mod_02_productos\src\blockchain\eventoNftContract.js

Que es	Modulo del backend (Node + ethers v6) que habla con el contrato EventoNFT en Ganache.
Que hace	Exporta 2 funciones: getNftConfig() lee nft-info.json y devuelve {address, abi}; verifyNFTComplete(eventoId, address) lee la cadena (getTokenId, getMetadata, ownerOf) y responde si hay NFT, su metadata y si la wallet indicada es la duena.
Para que sirve	Concentra toda la logica web3 del NFT en un solo lugar. La verificacion es de SOLO LECTURA: no gasta gas ni necesita firmar.

Codigo exacto (verificacion completa)

```
export async function verifyNFTComplete(eventoId, address) {
  const contrato = initContratoNFTLectura(); // provider de solo lectura
  const tokenId = await contrato.getTokenId(eventoId);
  if (tokenId.toString() === '0')
    return { hasNFT: false, tokenId: 0, metadata: null, ownership: null };
  const m = await contrato.getMetadata(tokenId); // metadata on-chain
  const owner = await contrato.ownerOf(tokenId); // dueno actual
  const isOwner = owner.toLowerCase() === String(address).toLowerCase();
  return { hasNFT: true, tokenId: tokenId.toString(),
    metadata: { eventoId: m.eventoId.toString(), name: m.name,
      lugar: m.lugar, precio: m.precio, hash: m.hash,
      timestamp: m.timestamp.toString() },
    ownership: { owner, isOwner } };
}
```

B4

Capa de datos eventoService.js (MODIFICADO)

ruta: E:\Portafolio\mod_02_productos\src\service\eventoService.js

Que es	La capa que ejecuta las consultas SQL contra MariaDB (tabla eventos).
Que hace	Se anadieron 2 metodos: updateNftToken(id, tokenId, owner) guarda el tokenId y el dueño inicial tras el mint; updateNftOwner(id, owner) actualiza solo el dueño tras una transferencia.
Para que sirve	Es donde el 'estado de la cadena' queda reflejado en la base de datos: así el dashboard sabe que evento ya tiene boleto y de quien es.

Codigo exacto

```
updateNftToken: async (id, nft_token_id, nft_owner = null) => {
  await pool.query(
    'UPDATE eventos SET nft_token_id = ?, nft_owner = ? WHERE id = ?',
    [nft_token_id, nft_owner, id]
  );
  return EventoService.findById(id);
},

updateNftOwner: async (id, nft_owner) => {
  await pool.query(
    'UPDATE eventos SET nft_owner = ? WHERE id = ?', [nft_owner, id]
  );
  return EventoService.findById(id);
},
```

B5

Controlador eventoController.js (MODIFICADO)

ruta: E:\Portafolio\mod_02_productos\src\controllers\eventoController.js

Que es	El controlador HTTP: recibe las peticiones REST, valida y llama al servicio o al modulo blockchain.
Que hace	Se anadieron 4 handlers: getNftConfig (entrega address+abi al frontend), updateNftToken (guarda tokenId+dueño, valida 400/404), transferNft (guarda el nuevo dueño tras transferir), verifyNftComplete (verificacion on-chain). Además toResponse ahora expone nftTokenId y nftOwner.
Para que sirve	Une el mundo HTTP (dashboard) con la BD y la blockchain, con validaciones claras de errores.

Codigo exacto (handler de transferencia)

```
transferNft: async (req, res) => {
  const { id } = req.params;
  const { owner } = req.body;
  if (!owner) return res.status(400).json({ error: 'Falta el campo owner' });
  const actual = await EventoService.findById(Number(id));
  if (!actual) return res.status(404).json({ error: 'Evento no encontrado' });
  if (!actual.nft_token_id)
    return res.status(400).json({ error: 'Este evento aun no tiene un NFT' });
  const updated = await EventoService.updateNftOwner(Number(id), owner);
  res.json(toResponse(updated));
},
```

B6

Rutas REST eventoRoutes.js (MODIFICADO)

RUTA: E:\Portafolio\mod_02_productos\src\routes\eventoRoutes.js

Que es	El archivo que define los endpoints de la API de eventos en Express.
Que hace	Se añadieron 4 rutas NFT. Las /nft/... van ANTES de /:id para que Express no confunda 'nft' con un id numerico.
Para que sirve	Son la puerta de entrada del frontend para todo el ciclo del boleto: config, guardar mint, guardar transferencia y verificar.

Codigo exacto

```
// Van ANTES de las rutas con :id
router.get('/nft/config', EventoController.getNftConfig);
router.get('/nft/verify-complete/:eventoId/:address',
  EventoController.verifyNftComplete);
...
router.put('/:id/nft', EventoController.updateNftToken); // tokenId tras mint
router.put('/:id/nft/transfer', EventoController.transferNft); // nuevo dueño
```

Metodo	Endpoint	Para que
GET	/api/eventos/nft/config	ABI + direccion del contrato NFT (para firmar en MetaMask)
GET	/api/eventos/nft/verify-complete/:eventoId/:address	Verificar tokenId + metadata + dueño
PUT	/api/eventos/:id/nft	Guardar { tokenId, owner } tras el mint
PUT	/api/eventos/:id/nft/transfer	Guardar { owner } tras transferir

B7

Esquema de la base de datos schema.sql (MODIFICADO)

RUTA: E:\Portafolio\mod_02_productos\src\db\schema.sql

Que es	El script SQL que define la tabla eventos en MariaDB (BD data_productos).
Que hace	Se añadieron 2 columnas: nft_token_id (el id del boleto dentro del contrato) y nft_owner (la wallet dueña actual). La migracion ya fue APLICADA a tu BD real.
Para que sirve	Sin estas columnas el mint fallaba al guardar (era el bug del 'tokenId que no se guardaba').

Codigo exacto (SQL)

```
nft_token_id INT NULL, -- tokenId del boleto NFT acunado
nft_owner VARCHAR(42) NULL, -- wallet dueña actual (cambia al transferir)

-- Migracion para BD ya existentes (ejecutar UNA sola vez):
ALTER TABLE eventos
ADD COLUMN nft_token_id INT NULL,
ADD COLUMN nft_owner VARCHAR(42) NULL;
```

B8

Prueba automatica verificar-nft.cjs (NUEVO)

RUTA: E:\Portafolio\mod_02_productos\scripts\verificar-nft.cjs

Que es	Un script de prueba que se ejecuta con npm run verify:nft sobre la red en memoria de Hardhat.
Que hace	Despliega EventoNFT, acuna un boleto, lee tokenId/metadata/dueno, hace una transferencia entre 2 cuentas (safeTransferFrom) y comprueba las 3 reglas de error (duplicado, campos vacios, direccion 0).
Para que sirve	Demuestra en segundos que el contrato funciona igual que lo hara en Ganache, sin depender de que Ganache este abierto.

Codigo exacto (prueba de transferencia)

```
const txT = await nftComprador['safeTransferFrom(address,address,uint256)'](
  comprador.address, organizador.address, tokenId);
await txT.wait();
const nuevoOwner = await nft.ownerOf(tokenId);
assert.equal(nuevoOwner, organizador.address); // el dueno CAMBIO OK
```

Capa web3 del NFT lib/nft.ts (NUEVO)

RUTA: E:\Portafolio\frontend_productos\lib\nft.ts

Que es	Modulo TypeScript del frontend (ethers v6) con toda la logica web3 del boleto NFT.
Que hace	3 funciones: mintEventoNFT() pide la config al backend, calcula el hash keccak256 y firma el mint con MetaMask; transferirNFT() firma safeTransferFrom validando antes que la cuenta conectada sea la duena; verificarNFT() consulta la verificacion al backend.
Para que sirve	El usuario firma TODO en su navegador: la clave privada nunca pasa por el servidor. La config del contrato llega en caliente desde /nft/config (tras cada deploy no hay que tocar nada).

Codigo exacto (transferencia entre cuentas)

```
export async function transferirNFT(tokenId: number, from: string, to: string) {
  if (!/^0x[0-9a-fA-F]{40}$/.test(to)) throw new Error('La direccion destino no es valida');
  const { address, abi } = await cargarNftConfig(); // GET /api/eventos/nft/config
  const provider = new BrowserProvider(window.ethereum!);
  const signer = await provider.getSigner(); // abre MetaMask
  const contrato = new Contract(address, abi, signer);
  const ownerActual = (await contrato.ownerOf(tokenId)) as string;
  if (ownerActual.toLowerCase() !== from.toLowerCase())
    throw new Error('La cuenta conectada no es la duena del boleto');
  // ERC-721: safeTransferFrom(from, to, tokenId)
  const tx = await contrato['safeTransferFrom(address,address,uint256)'](from, to, tokenId);
  const receipt = await tx.wait(); // se mina en Ganache
  return { txHash: receipt.hash };
}
```

Codigo exacto (nucleo del mint)

```
const hash = solidityPackedKeccak256(['uint256','string','string'],
  [evento.id, evento.name, evento.lugar]);
const tx = await contrato.mintEventoNFT(evento.id, evento.name, evento.lugar,
  precio, hash, account);
await tx.wait();
const tokenId = Number(await contrato.getTokenId(evento.id));
return { txHash: tx.hash, tokenId };
```


F2

Cliente de la API lib/api.ts (MODIFICADO)

RUTA: E:\Portafolio\frontend_productos\lib\api.ts

Que es	El modulo que centraliza todas las llamadas fetch del dashboard hacia la API del backend.
Que hace	Se anadieron 3 funciones: getNftConfig() , updateNftToken(id, tokenId, owner) y transferNftOwner(id, owner) .
Para que sirve	Separa la UI de la red: los componentes solo llaman funciones tipadas; las URLs y errores viven aqui.

Codigo exacto

```
export async function updateNftToken(id: number, tokenId: number, owner?: string) {
  const res = await fetch(`${BASE}/${id}/nft`, {
    method: 'PUT', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ tokenId, owner }),
  });
  ...
}
export async function transferNftOwner(id: number, owner: string) {
  const res = await fetch(`${BASE}/${id}/nft/transfer`, {
    method: 'PUT', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ owner }),
  });
  ...
}
```

F3

Tipos lib/types.ts (MODIFICADO)

RUTA: E:\Portafolio\frontend_productos\lib\types.ts

Que es	Las interfaces TypeScript del dominio (Evento, CreateEventoDTO).
Que hace	El tipo Evento gana 2 campos opcionales: nftTokenId y nftOwner .
Para que sirve	Permite que la tabla sepa (con tipado seguro) si un evento ya tiene boleto y de quien es, para decidir que botones mostrar.

Codigo exacto

```
txHash?: string | null; // prueba on-chain: hash de la transaccion
onchainId?: number | null; // id del evento dentro del contrato Events
nftTokenId?: number | null; // tokenId del boleto NFT acunado (EventoNFT)
nftOwner?: string | null; // wallet duena actual del NFT (cambia al transferir)
```

F4

Dialogos lib/swal.ts (MODIFICADO)

RUTA: E:\Portafolio\frontend_productos\lib\swal.ts

Que es	Helpers de SweetAlert2 con el tema visual del dashboard (alertas, toasts, confirmaciones).
Que hace	Se anadio promptText() : un dialogo con campo de texto que pide/confirma la wallet destino antes de transferir (viene prellenado con la Cuenta B).

Para que sirve

Da una forma consistente y bonita de capturar la direccion destino sin construir un formulario nuevo.

Codigo exacto (resumen)

```
export async function promptText(title: string, opts = {}) {
  const r = await modal.fire({
    title, html: opts.html, input: 'text',
    inputValue: opts.value ?? '',          // prellenado: cuenta B 0xada5...
    showCancelButton: true,
    confirmButtonText: opts.confirmText ?? 'Continuar',
  });
  return r.isConfirmed ? (r.value as string).trim() : null;
}
```

Tabla del dashboard EventsTable.tsx (MODIFICADO)

RUTA: E:\Portafolio\frontend_productos\components\dashboard\EventsTable.tsx

Que es	El componente React que pinta la tabla de eventos (escritorio y movil) con sus acciones.
Que hace	Se anadieron los 3 botones NFT y sus handlers: Mint NFT (icono chispas) si el evento no tiene boleto; Validar NFT (icono insignia) y Transferir (icono enviar) si ya lo tiene. El mint guarda tokenId + dueño; la transferencia firma en MetaMask y luego persiste el nuevo dueño.
Para que sirve	Es la cara visible de todo el flujo: cada boton dispara el ciclo completo navegador -> MetaMask -> Ganache -> API -> MariaDB.

Codigo exacto (handlers, resumido)

```
// Cuenta de Ganache sugerida como destino al transferir (importada en MetaMask)
const DESTINO_SUGERIDO = '0xada53769c1679cc8A927806b7417Cc7B297Bcd9d';

const mintNFT = async (ev: Evento) => {
  const { txHash, tokenId } = await mintEventoNFT(ev, account);
  await updateNftToken(ev.id, tokenId, account); // guarda tokenId + dueño
  onRefresh();
};

const transferNFT = async (ev: Evento) => {
  const destino = await promptText('Transferir boleto NFT',
    { value: DESTINO_SUGERIDO, confirmText: 'Transferir' });
  if (!destino) return;
  const { txHash } = await transferirNFT(ev.nftTokenId, account, destino);
  await transferNftOwner(ev.id, destino); // guarda el nuevo dueño
  onRefresh();
};
```

Codigo exacto (los 3 botones, version escritorio)

```
{!ev.nftTokenId ? (
  <button onClick={() => mintNFT(ev)} title="Acunar boleto NFT">
    <Sparkles size={14} />      {/* boton MINT */}
  </button>
) : (
  <>
    <button onClick={() => verNFT(ev)} title={`Validar boleto NFT #${ev.nftTokenId}`}>
      <BadgeCheck size={14} />      {/* boton VALIDAR */}
    </button>
    <button onClick={() => transferNFT(ev)} title="Transferir a otra wallet">
      <Send size={14} />      {/* boton TRANSFERIR */}
    </button>
  </>
)}
```

F6

Pagina del dashboard page.tsx (MODIFICADO)

RUTA: E:\Portafolio\frontend_productos\app\dashboard\page.tsx

Que es	La pagina principal del dashboard (App Router de Next.js).
Que hace	Ahora pasa la wallet conectada (account , del hook useWallet) como prop a EventsTable.
Para que sirve	Los botones NFT necesitan saber QUE cuenta esta conectada para acunar a su nombre y para validar la propiedad.

Codigo exacto

```
const { account, connect, connecting } = useWallet();
...
<EventsTable eventos={eventos} onRefresh={() => fetchData(true)} account={account} />
```

F7

package.json - fix del puerto (MODIFICADO)

RUTA: E:\Portafolio\frontend_productos\package.json

Que es	La configuracion de scripts npm del frontend.
Que hace	Se fijo el puerto 3001 en dev y start. Antes 'next dev' tomaba el 3000 por defecto y CHOCABA con el backend.
Para que sirve	Ese choque era el bug 'no se puede conectar a la API': el proxy /api apuntaba al propio Next en vez del backend. Con puertos fijos ya no puede repetirse.

Codigo exacto

```
"scripts": {
  "dev": "next dev -p 3001",
  "build": "next build",
  "start": "next start -p 3001",
  "lint": "eslint"
}
```

F8

.env.local - direccion del contrato (MODIFICADO)

RUTA: E:\Portafolio\frontend_productos\.env.local

Que es	Variables de entorno publicas del frontend.
Que hace	Se actualizo NEXT_PUBLIC_CONTRACT_ADDRESS a la direccion VIGENTE del contrato Events desplegado en Ganache (la anterior apuntaba a un despliegue viejo).
Para que sirve	El boton de 'Emitir on-chain' (rayo) firma contra esta direccion; si esta desactualizada, la emision falla. La del NFT NO se configura aqui: llega sola desde /nft/config.

Codigo exacto

```
NEXT_PUBLIC_API_URL=http://localhost:3000
NEXT_PUBLIC_CONTRACT_ADDRESS=0x2345b1871CEdf0ff4d56a7958af67E533742aA35
```

C1

Correcciones aplicadas a la base de datos (MariaDB)

BD: **data_productos** en localhost:3306 - tabla **eventos**. Acceso rapido por consola:

Comandos para inspeccionar

```
mysql -u root -proot data_productos
SHOW COLUMNS FROM eventos;
SELECT id, name, tx_hash, onchain_id, nft_token_id, nft_owner FROM eventos;
```

Problema encontrado	Correccion aplicada
Faltaban las columnas nft_token_id y nft_owner (el mint no podia guardar el tokenId).	ALTER TABLE eventos ADD COLUMN nft_token_id INT NULL, ADD COLUMN nft_owner VARCHAR(42) NULL; (YA APLICADO)
Los tx_hash / onchain_id apuntaban a un contrato Events VIEJO (0x521420...) con datos desactualizados: la validacion siempre daba 'difiere'.	Se re-registraron los 3 eventos en el contrato vigente (0x2345b1...) con los datos actuales de la BD y se actualizaron tx_hash + onchain_id. Validacion: 3/3 COINCIDEN.
Eventos 2 y 3 tenian NFT on-chain (tokens 2 y 1) pero la BD tenia NULL (se mintearon antes de la migracion).	Sincronizacion BD <-> cadena: se guardo tokenId y dueño reales leyendo getTokenId/ownerOf del contrato.

Estado final: evento 1 (sin NFT, listo para Mint) - evento 2 (token 2) - evento 3 (token 1), ambos de la Cuenta A. Todo coincide con la cadena.

C2

Como ponerlo todo en marcha (guia rapida)

Pasos exactos

```
REM 1) Ganache GUI abierto (http://127.0.0.1:7545, chainId 1337)
REM 2) Backend
cd E:\Portafolio\mod_02_productos
npm run dev                REM API en http://localhost:3000

REM 3) Frontend (otra terminal)
cd E:\Portafolio\frontend_productos
npm run dev                REM dashboard en http://localhost:3001

REM (solo si reinicias Ganache) redespiega y pega la nueva direccion:
cd E:\Portafolio\mod_02_productos
npm run deploy             REM actualiza contract-info.json y nft-info.json
REM -> copiar direccion de Events a frontend_productos\.env.local
```

Flujo de demo: Conectar wallet (Cuenta A) -> Mint NFT en el evento 1 -> Validar NFT -> Transferir (destino prellenado: Cuenta B 0xada5...) -> cambiar a Cuenta B en MetaMask -> Validar de nuevo: 'este boleto te pertenece'.

Fin de la guía - Cambios Frontend y Backend con rutas y código exacto